SENAI Serviço Nacional de Aprendizagem Industrial

ENGENHARIA DE SOFTWARE

ESTRUTURA DE DADOS II

DESAFIO: índice e despacho integrados

Prof. Luciano Lima

Contexto do desafio

A esteira da empresa de logística chega ao fim de um turno. Dois problemas precisam ser resolvidos ao mesmo tempo, e sua equipe de engenharia foi chamada para decidir e implementar a arquitetura de dados por trás dos dois:

1. **Índice de produtos**: o sistema continua recebendo buscas de produtos por SKU durante e depois do fechamento, o índice não pode parar.
2. **Fechamento da esteira de despacho**: o lote de pedidos pendentes, acumulado ao longo do turno, precisa ser processado de uma vez por ordem de prioridade, mas novos pedidos urgentes continuam chegando depois do fechamento, e também precisam ser atendidos pela ordem certa, sem reprocessar o lote inteiro a cada chegada.

Vocês vão decidir a estrutura de dados certa para cada parte, implementa-las, e justificar tecnicamente as escolhas, exatamente como fariam na defesa de arquitetura do projeto integrador.

O que sua equipe precisa entregar

Parte 1: decisão de arquitetura do índice (AVL ou rubro-negra)

O índice de produtos vai ser consultado por SKU continuamente, mas também recebe inserções e remoções (entrada e saída de produtos do catálogo). Decidam entre árvore AVL e árvore rubro-negra para esse índice, e registrem a decisão na ficha de decisão (seção seguinte). A justificativa precisa comparar as duas estruturas, não basta dizer que ambas são O(log n).

Ficha de decisão de arquitetura (preencher e entregar junto com o código)

Estrutura escolhida: AVL ou rubro-negra?

Qual é o padrão de uso esperado para este índice: mais busca ou mais escrita (inserção/remoção)? Por quê?

Em uma frase: por que a estrutura escolhida se encaixa melhor nesse padrão do que a outra opção?

Parte 2: fechamento do lote com heapsort

Implementem heapsort para ordenar, de uma só vez, o vetor de pedidos pendentes do turno (fornecido na seção de dataset) por prioridade decrescente: a urgência mais alta primeiro.

Parte 3: fila de despacho pós-fechamento

Depois do fechamento, os pedidos que chegarem precisam ser atendidos por prioridade, um de cada vez, sem reprocessar o lote inteiro a cada chegada. Usem o vetor já ordenado pelo heapsort da parte 2 como ponto de partida para montar uma fila de prioridade (heap) que aceite a inserção dos novos pedidos urgentes listados no dataset, mantendo sempre a próxima remoção correta.

As partes 2 e 3 precisam conversar entre si: não vale implementar heapsort e a fila de prioridade como dois exercícios isolados, o heap final da fila de despacho tem que nascer do resultado do heapsort do lote, não ser reconstruído do zero.

Parte 4: índice funcional

Implementem a estrutura de índice escolhida na parte 1 (AVL ou rubro-negra), com inserção e busca por SKU, usando o dataset de produtos fornecido.

Parte 5: justificativa técnica

Entreguem, junto com o código, um texto com:

1. A ficha de decisão preenchida (parte 1)
2. A análise de complexidade ponta a ponta da solução completa (índice + fechamento do lote + fila de despacho);
3. E uma observação sobre o que mudaria na arquitetura se o volume de pedidos por turno crescesse de milhares para milhões.

Dataset fornecido

Produtos para o índice (inserir todos, nesta ordem, na estrutura escolhida):

```
SKU-3010, SKU-1045, SKU-5520, SKU-2230, SKU-4190,
SKU-1800, SKU-6005, SKU-2990, SKU-3770, SKU-4450
```

Lote de pedidos pendentes ao fechar o turno, cada par é (id do pedido, urgência de 1 a 10, sendo 10 a mais urgente):

```
(P101, 6), (P102, 9), (P103, 2), (P104, 7), (P105, 4),
(P106, 10), (P107, 3), (P108, 8), (P109, 5), (P110, 1)
```

Pedidos urgentes que chegam depois do fechamento, nesta ordem (inserir na fila de prioridade já construída, um a um):

(P111, 9), (P112, 10), (P113, 6)

Critérios de pronto

- O lote de pedidos pendentes sai do heapsort em ordem decrescente de urgência.
- A fila de prioridade pós-fechamento aceita os três pedidos novos e sempre remove corretamente o de maior urgência, incluindo entre pedidos empatados e os recém-inseridos.
- O índice de produtos responde corretamente a uma busca por SKU após a inserção de todos os produtos do dataset.
- A ficha de decisão de arquitetura está preenchida e a justificativa compara as duas estruturas, não apenas descreve a escolhida.
- A análise de complexidade cobre as três partes da solução, não apenas uma delas.

Formato de entrega

- Código-fonte Java das três partes (índice, heapsort do lote, fila de prioridade), compilável e testado.
- Ficha de decisão de arquitetura preenchida e a justificativa técnica (parte 5), em um único arquivo de texto ou documento.
- Identificação de todos os integrantes do grupo em ambos os arquivos.

Esta atividade irá compor parte dos 30% da nota da N2.